



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по учебной практике

УП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции
программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Смирнова Виктория Александровна
(подпись) (фамилия, имя, отчество)

Группы ИПо-32

Руководитель практической подготовки от профильной организации

Преподаватель (должность)

(подпись) Берестов Дмитрий Борисович
(фамилия, имя, отчество)

МП

«26» июня 2026 года

Руководитель практической подготовки от института

(подпись) Берестов Дмитрий Борисович
(фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	2
РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ.....	4
1.1. Сбор информации о существующем состоянии продукта:.....	5
1.2 Разработка технического задания.....	6
РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ.....	6
2.1. Процесс разработки программного обеспечения.....	7
РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ.....	11
3.1. Процесс отладки.....	11
РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ.....	12
4.1. Процесс тестирования.....	12
РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ.....	13
5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.....	13
ЗАКЛЮЧЕНИЕ.....	14
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....	16

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Учебная практика направлена на формирование у обучающихся первоначальных практических профессиональных умений в рамках модулей ППСЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем

В настоящей работе представлен отчет о прохождении учебной практики студента-практиканта. Основными видами деятельности являлись: Разработка программного обеспечения; выполнение отладки программного модуля с использованием специализированных программных средств; осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; уметь производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент;

Выполнять интеграцию модулей в программное обеспечение;

Выполнять отладку программного модуля с использованием специализированных программных средств;

Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.

Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;

Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

Планировать и реализовывать собственное профессиональное и личностное развитие;

Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами

Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.

Использовать информационные технологии в профессиональной деятельности

Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ

1.1. Сбор информации о существующем состоянии продукта:

Целью разработки являлось создание приложения для управления задачами пользователя. Программное обеспечение должно обеспечивать удобное хранение информации о задачах, возможность их создания, изменения, удаления и поиска.

Состояние системы «AS IS»

До автоматизации пользователь самостоятельно вёл список задач. Каждая задача хранилась в виде отдельной записи. Для поиска необходимой информации приходилось просматривать весь список вручную. Изменение статуса выполнения задач также выполнялось пользователем без дополнительных инструментов контроля.

Основные действия процесса:

- создание записи о задаче;
- хранение информации;
- поиск задачи;
- изменение данных;
- удаление неактуальных записей;
- контроль выполнения задач.

Состояние системы «TO BE»

После внедрения программного продукта все операции выполняются автоматически. Пользователь работает через графический интерфейс приложения и может создавать новые задачи, редактировать существующие записи, выполнять поиск и фильтрацию данных.

Основные действия процесса:

- создание новой задачи через форму ввода;
- сохранение данных в XML-файл;
- просмотр списка задач;
- поиск по названию и описанию;
- фильтрация по статусу;
- сортировка по приоритету и сроку выполнения;
- получение статистики выполнения задач.

Сравнительный анализ показал, что внедрение программного продукта позволяет значительно сократить время обработки информации и повысить удобство работы пользователя.

1.2 Разработка технического задания

Основные требования к программному продукту: ‘

- создание новых задач;
- редактирование задач;
- удаление задач;
- поиск задач;
- фильтрация по статусу;
- сохранение данных в XML-файл;
- загрузка данных из XML-файла;
- отображение статистики;
- наличие графического интерфейса пользователя.

РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

2.1. Процесс разработки программного обеспечения

Для разработки приложения была выбрана итерационная модель разработки. Данная модель позволяет последовательно создавать отдельные компоненты системы, проверять их работоспособность и при необходимости вносить изменения на каждом этапе разработки.

Разработка программного обеспечения осуществлялась в среде Visual Studio 2022 на языке программирования C#. Для создания пользовательского интерфейса использовалась технология WPF (Windows Presentation Foundation).

Программный продукт состоит из нескольких взаимосвязанных модулей. Основным объектом системы является класс Task, предназначенный для хранения информации о задаче. Класс содержит идентификатор задачи, название, описание, приоритет, срок выполнения и статус.

Для ограничения допустимых значений были созданы перечисления Priority и Status. Использование перечислений позволяет избежать ошибок при вводе данных и обеспечивает единообразное хранение информации.

Для хранения данных был разработан класс XmlContext. Данный класс обеспечивает загрузку задач из XML-файла и сохранение изменений после выполнения операций над данными.

Основная логика приложения реализована в классе TaskService. Класс выполняет операции создания, редактирования, удаления, поиска и фильтрации задач, а также формирует статистическую информацию.

Для ввода, просмотра и редактирования данных были разработаны специальные формы.

Окно TaskWindow позволяет создавать новые задачи и изменять существующие записи. (см рис 2,3,4)

Окно TaskListWindow обеспечивает просмотр списка задач, сортировку и фильтрацию данных.

Окно StatisticsWindow предназначено для отображения статистики по задачам.

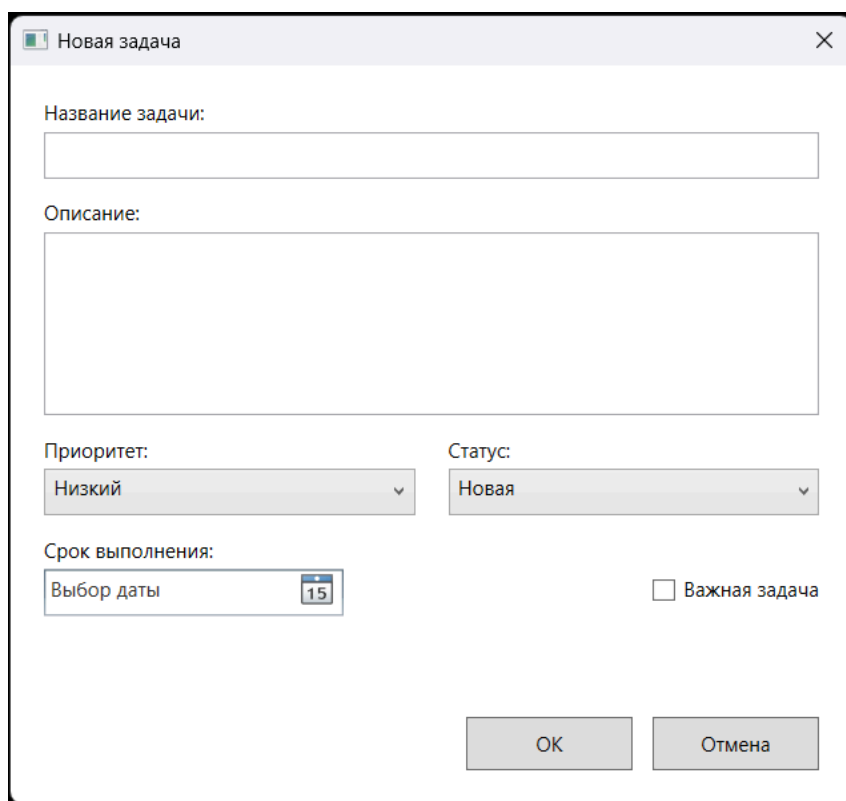


рисунок 2-пустое окно создания новой задачи

Редактирование задачи

Название задачи:
Гамма

Описание:
Гамма

Приоритет: Высокий

Статус: Завершена

Срок выполнения: 05.07.2026 15

☒ Важная задача

OK Отмена

рисунок 3-заполненное окно редактирования задачи

Редактирование задачи

Название задачи:

Описание:
Гамма

Приоритет: Высокий

Статус: Завершена

Срок выполнения: 05.07.2026 15

☒ Важная задача

Название задачи не заполнено

OK Отмена

рисунок 4-окно редактирования с некорректными данными

РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ

3.1. Процесс отладки

После завершения разработки приложения была выполнена отладка программного обеспечения. Основной целью отладки являлось выявление и устранение ошибок, возникающих при работе программы.

Отладка проводилась в среде Visual Studio 2022 с использованием встроенных средств отладки.

На первом этапе была проверена работа класса XmlContext. Выполнялось тестирование загрузки данных из XML-файла и сохранения изменений после выполнения операций над задачами. Проверялась корректность сериализации и десериализации объектов класса Task.

На следующем этапе была выполнена отладка класса TaskService. Проверялась корректность работы методов добавления, изменения и удаления задач. Особое внимание уделялось правильности работы поиска и фильтрации данных.

Также была проведена проверка пользовательского интерфейса. Выполнялось тестирование окон TaskListWindow, TaskWindow и StatisticsWindow. Контролировалась корректность отображения данных, работа кнопок управления и переходов между окнами приложения.

РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1. Процесс тестирования

После завершения разработки и отладки было проведено тестирование программного обеспечения. Основной целью тестирования являлась проверка корректности работы всех функций приложения.

В ходе тестирования были разработаны тестовые сценарии для проверки следующих возможностей программы:

- создание новой задачи;
- редактирование существующей задачи;
- удаление задачи;
- просмотр списка задач;
- поиск задач по названию;
- поиск задач по описанию;
- фильтрация по статусу;
- сортировка по приоритету;
- сортировка по сроку выполнения;
- сохранение данных в XML-файл;
- загрузка данных из XML-файла;
- формирование статистики.

В рамках модульного тестирования были разработаны тесты для проверки операций добавления, удаления, редактирования и поиска задач. Также проверялась корректность сохранения данных и формирования статистики.

Проведённое тестирование подтвердило корректность работы всех компонентов приложения и отсутствие критических ошибок в работе программного обеспечения.

РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ

5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

Инспектирование программного кода выполнялось с целью проверки качества реализации программного продукта и соответствия кода принятым стандартам разработки.

Проверка проводилась для основных компонентов приложения: классов Task, TaskService, XmlContext, а также элементов пользовательского интерфейса WPF-приложения.

В процессе инспектирования использовались следующие виды проверки:

- визуальный анализ исходного кода;
- автоматическая проверка средствами Visual Studio;
- анализ структуры проекта.

Для автоматизированной проверки использовались встроенные анализаторы среды Visual Studio, позволяющие выявлять потенциальные ошибки и нарушения стандартов программирования.

Проведённое инспектирование показало, что разработанный программный продукт соответствует требованиям качества и может использоваться для решения поставленной задачи.

ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики с 8 по 28 июня 2024 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

1. Выполнен анализ требований к программному обеспечению и разработано техническое задание на создание приложения для управления задачами.
2. Спроектирована структура программного продукта и разработаны основные классы приложения.
3. Реализован механизм хранения и обработки данных с использованием XML-файлов.
4. Разработан пользовательский интерфейс приложения на основе технологии WPF.
5. Реализованы функции создания, редактирования, удаления, поиска, фильтрации и сортировки задач.
6. Выполнены отладка и тестирование программного обеспечения.
7. Разработаны модульные тесты для проверки работоспособности основных функций приложения.
8. Проведено инспектирование программного кода на предмет соответствия стандартам кодирования.

По окончании практики был получен практический опыт разработки требований к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного

обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201–78. Техническое задание. Требования к содержанию и оформлению. – М.: Издательство стандартов, 1978.
2. ГОСТ 34.602–89. Техническое задание на создание автоматизированной системы. – М.: Издательство стандартов, 1989.
3. Троелсен Э., Джепикс Ф. Язык программирования C# 10 и платформа .NET 6. – СПб.: Питер, 2023. – 944 с.
4. Рихтер Д. CLR via C#. Программирование на платформе Microsoft .NET Framework. – СПб.: Питер, 2022. – 896 с.
5. Шилдт Г. C# 10. Полное руководство. – М.: Вильямс, 2023. – 960 с.
6. Прайс М. C# 10 и .NET 6. Современная кроссплатформенная разработка. – СПб.: Питер, 2023. – 820 с.
7. Visual Studio Documentation [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/visualstudio/> (дата обращения: 22.06.2026).
8. WPF Documentation [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/> (дата обращения: 22.06.2026).
9. Microsoft Learn. Документация по C# [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения: 22.06.2026).
10. Microsoft Learn. XML Serialization in .NET [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/standard/serialization/xml-serialization> (дата обращения: 23.06.2026).

ToDo\App.xaml

```
<Application x:Class="ToDo.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="clr-namespace:ToDo">
    <Application.Resources>
    </Application.Resources>
</Application>
```

ToDo\App.xaml.cs

```
using System;
using System.IO;
using System.Text.Json;
using System.Windows;
using ToDo.Data.Context;
using ToDo.Data.Services;
using ToDo.Windows;
namespace ToDo
{
    /// <summary>
    /// Interaction logic for App.xaml
    /// </summary>
    public partial class App : Application
    {
        static string _xmlFile { get; set; }
        public TaskService service { get; set; }
        public App()
        {
            SetXmlFilePath();
            service = new TaskService(new XmlContext(_xmlFile));
            TaskListWindow orgsView = new TaskListWindow(service);
            orgsView.Show();
        }
    }
}
```

```

    }
    private void SetXmlFilePath()
    {
        var settingsPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "settings.json");
        var json = File.ReadAllText(settingsPath);
        using var document = JsonDocument.Parse(json);
        if (!document.RootElement.TryGetProperty("XmlFile", out var xmlFileElement) ||
            xmlFileElement.ValueKind != JsonValueKind.String ||
            string.IsNullOrEmpty(_xmlFile = xmlFileElement.GetString()))
        {
            throw new InvalidDataException("Указан неправильный путь до xml файла.");
        }
    }
}

```

ToDo\Windows\TaskWindow.xaml

```

<Window x:Class="ToDo.Windows.TaskWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:local="clr-namespace:ToDo.Windows"
    xmlns:local0="clr-namespace:ToDo"
    xmlns:local1="clr-namespace:ToDo.Helpers"
    mc:Ignorable="d" Title="Задача"
    Height="480" Width="520"
    ResizeMode="NoResize">
    <Window.Resources>
        <local1:PriorityConverter x:Key="PrConv" />
        <local1:StatusConverter x:Key="StConv" />
    </Window.Resources>
    <Window.CommandBindings>

```

```

        <CommandBinding Command="local0:WindowCommands.TaskOk" Executed="BtnOk_Click"/>
        <CommandBinding Command="local0:WindowCommands.TaskCancel"
Executed="BtnCancel_Click"/>
    </Window.CommandBindings>
    <Grid Margin="20">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
        <TextBlock Grid.Row="0" Text="Название задачи:" Margin="0 0 0 4"/>
        <TextBox Grid.Row="1" x:Name="NameField" Height="28" Margin="0 0 0 12"/>
        <TextBlock Grid.Row="2" Text="Описание:" Margin="0 0 0 4"/>
        <TextBox Grid.Row="3" x:Name="DescriptionField" Height="110"
            TextWrapping="Wrap" AcceptsReturn="True"
            VerticalScrollBarVisibility="Auto" Margin="0 0 0 12"/>
        <Grid Grid.Row="4" Margin="0 0 0 12">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*/>
                <ColumnDefinition Width="20"/>
                <ColumnDefinition Width="*/>
            </Grid.ColumnDefinitions>
            <StackPanel Grid.Column="0">
                <TextBlock Text="Приоритет:" Margin="0 0 0 4"/>
                <ComboBox x:Name="PriorityField" Height="28">
                    <ComboBox.ItemTemplate>
                        <DataTemplate>
                            <TextBlock Text="{Binding Converter={StaticResource PrConv}}"/>

```

```

        </DataTemplate>
    </ComboBox.ItemTemplate>
</ComboBox>
</StackPanel>
<StackPanel Grid.Column="2">
    <TextBlock Text="Статус:" Margin="0 0 0 4"/>
    <ComboBox x:Name="StatusField" Height="28">
        <ComboBox.ItemTemplate>
            <DataTemplate>
                <TextBlock Text="{Binding Converter={StaticResource StConv}}"/>
            </DataTemplate>
        </ComboBox.ItemTemplate>
    </ComboBox>
</StackPanel>
</Grid>
<Grid Grid.Row="5" Margin="0 0 0 16">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="Auto" />
    </Grid.ColumnDefinitions>
    <StackPanel Grid.Column="0">
        <TextBlock Text="Срок выполнения:" Margin="0 0 0 4"/>
        <DatePicker x:Name="DateField" Height="28" Width="180" HorizontalAlignment="Left"/>
    </StackPanel>
    <CheckBox Grid.Column="1" x:Name="ImportantField"
        Content="Важная задача"
        VerticalAlignment="Bottom"
        Margin="30 0 0 8"/>
</Grid>
<TextBlock Grid.Row="6" x:Name="ErrorField"
    Foreground="Red" FontWeight="SemiBold" TextWrapping="Wrap"
    Margin="0 4 0 8" Visibility="Collapsed"/>

```

```

        <StackPanel Grid.Row="7" Orientation="Horizontal" HorizontalAlignment="Right">
            <Button Command="local0:WindowCommands.TaskOk" Content="OK" Width="100"
Height="32" Margin="0 0 12 0"/>
            <Button Command="local0:WindowCommands.TaskCancel" Content="Отмена" Width="100"
Height="32"/>
        </StackPanel>
    </Grid>
</Window>

```

ToDo\Windows\TaskWindow.xaml.cs

```

using System.Windows;
using System.Windows.Input;
using ToDo.Data.Models;
using ToDo.Data.Services;
namespace ToDo.Windows
{
    /// <summary>
    /// Логика взаимодействия для TaskWindow.xaml
    /// </summary>
    public partial class TaskWindow : Window
    {
        private readonly TaskService _service;
        private Data.Models.Task? _task;
        private readonly bool _editFlag;
        public TaskWindow(TaskService service, Data.Models.Task? taskToEdit = null)
        {
            InitializeComponent();
            _service = service;
            _task = taskToEdit;
            _editFlag = taskToEdit != null;
            Title = _editFlag ? "Редактирование задачи" : "Новая задача";
            DataContext = _task;
            FillFields();
        }
    }
}

```

```

private void FillFields()
{
    PriorityField.ItemsSource = Enum.GetValues(typeof(Priority));
    PriorityField.SelectedItem = Priority.Low;
    StatusField.ItemsSource = Enum.GetValues(typeof(Status));
    StatusField.SelectedItem = Status.New;
    if (_editFlag)
    {
        PriorityField.SelectedItem = _editFlag ? _task.Priority : Priority.Low;
        StatusField.SelectedItem = _editFlag ? _task.Status : Status.New;
        NameField.Text = _task.Name;
        DescriptionField.Text = _task.Description;
        DateField.SelectedDate = _task.Date;
        ImportantField.IsChecked = _task.ImportantFlag;
    }
}

private void BtnOk_Click(object sender, ExecutedRoutedEventArgs e)
{
    ErrorField.Visibility = Visibility.Collapsed;
    SaveTask();
}

private void BtnCancel_Click(object sender, ExecutedRoutedEventArgs e)
{
    DialogResult = false;
    Close();
}

private void SaveTask()
{
    if (!_editFlag)
    {
        _task = new Data.Models.Task();
    }
}

```

```

        _task.Name = NameField.Text.Trim();
        _task.Description = DescriptionField.Text.Trim();
        _task.Priority = (Priority)PriorityField.SelectedItem;
        _task.Status = (Status)StatusField.SelectedItem;
        _task.Date = DateField.SelectedDate ?? DateTime.Now.AddDays(1);
        _task.ImportantFlag = ImportantField.IsChecked == true;
        if (string.IsNullOrEmpty(_task.Name))
        {
            ErrorField.Text = "Название задачи не заполнено";
            ErrorField.Visibility = Visibility.Visible;
            return;
        }
        bool res = _editFlag ? _service.EditTask(_task) : _service.SaveTask(_task);
        if (res)
        {
            DialogResult = true;
            Close();
        }
        else
        {
            ErrorField.Text = "Ошибка сохранения";
            ErrorField.Visibility = Visibility.Visible;
        }
    }
}

```

ToDo\Windows\TaskListWindow.xaml

<Window x:Class="ToDo.Windows.TaskListWindow"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

xmlns:d="http://schemas.microsoft.com/expression/blend/2008"

xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"

```

xmlns:local="clr-namespace:ToDo.Windows"
xmlns:local0="clr-namespace:ToDo"
xmlns:local1="clr-namespace:ToDo.Helpers"
mc:Ignorable="d"
Title="Список задач" Height="550" Width="1100">
<Window.Resources>
    <local1:DateTimeConverter x:Key="DTConv"/>
    <local1:PriorityConverter x:Key="PrConv" />
    <local1:StatusConverter x:Key="StConv" />
</Window.Resources>
<Window.CommandBindings>
    <CommandBinding Command="local0:WindowCommands.NewTask"
Executed="NewTask_Executed"/>
    <CommandBinding Command="local0:WindowCommands.EditTask"
Executed="EditTask_Executed"/>
    <CommandBinding Command="local0:WindowCommands.DeleteTask"
Executed="DeleteTask_Executed"/>
    <CommandBinding Command="local0:WindowCommands.ClearFilters"
Executed="ClearFilters_Executed"/>
    <CommandBinding Command="local0:WindowCommands.GetStatistics"
Executed="GetStatistics_Executed"/>
</Window.CommandBindings>
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/>
    </Grid.RowDefinitions>
    <DockPanel Grid.Row="0">
        <Menu DockPanel.Dock="Top" Background="#7FFF00">
            <MenuItem Header="Задачи">
                <MenuItem Header="Новая задача" Command="local0:WindowCommands.NewTask"/>
                <MenuItem Header="Статистика" Command="local0:WindowCommands.GetStatistics"/>
            </MenuItem>
        </Menu>

```



```

</DockPanel>

<Grid Grid.Row="1" Margin="10">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="200"/>
        <ColumnDefinition Width="*/>
    </Grid.ColumnDefinitions>
    <GroupBox Header="Фильтры" Margin="0 0 10 0" Padding="10">
        <StackPanel>
            <TextBlock Text="Поиск:" Margin="0 8 0 4"/>
            <TextBox x:Name="SearchField" TextChanged="FiltersChanged" Height="26"/>
            <TextBlock Text="Статус:" Margin="0 12 0 4"/>
            <ComboBox x:Name="StatusField" Height="26" SelectionChanged="FiltersChanged">
                <ComboBoxItem Content="Все"/>
                <ComboBoxItem Content="Новые"/>
                <ComboBoxItem Content="В процессе"/>
                <ComboBoxItem Content="Завершены"/>
            </ComboBox>
            <TextBlock Text="Приоритет:" Margin="0 12 0 4"/>
            <ComboBox x:Name="PriorityField" Height="26" SelectionChanged="FiltersChanged">
                <ComboBoxItem Content="Все"/>
                <ComboBoxItem Content="Низкий"/>
                <ComboBoxItem Content="Средний"/>
                <ComboBoxItem Content="Высокий"/>
            </ComboBox>
            <TextBlock Text="Дата:" Margin="0 12 0 4"/>
            <DatePicker x:Name="DateField" Height="26" SelectedDateChanged="FiltersChanged"/>
            <CheckBox x:Name="ImportantOnlyField" Content="Только важные" Margin="0 12 0 0"
Height="24"
                Checked="FiltersChanged" Unchecked="FiltersChanged"/>
            <Button Content="Очистить все фильтры" Margin="0 20 0 0" Height="30"
Command="local0:WindowCommands.ClearFilters"/>
        </StackPanel>
    </GroupBox>

```

```

        <DataGrid x:Name="TasksList" Grid.Column="1" AutoGenerateColumns="False"
IsReadOnly="True"

        SelectionMode="Single" MouseDoubleClick="TasksList_MouseDoubleClick"
ItemsSource="{Binding}">

        <DataGrid.ContextMenu>

        <ContextMenu>

        <MenuItem Header="Изменить" Command="local0:WindowCommands.EditTask" />

        <MenuItem Header="Удалить" Command="local0:WindowCommands.DeleteTask" />

        </ContextMenu>

        </DataGrid.ContextMenu>

        <DataGrid.Columns>

        <DataGridTemplateColumn Header="" Width="50" SortMemberPath="ImportantFlag">

        <DataGridTemplateColumn.CellTemplate>

        <DataTemplate>

        <TextBlock Text="★" HorizontalAlignment="Center" VerticalAlignment="Center"

        FontSize="18" Foreground="Orange">

        <TextBlock.Style>

        <Style TargetType="TextBlock">

        <Setter Property="Visibility" Value="Collapsed"/>

        <Style.Triggers>

        <DataTrigger Binding="{Binding ImportantFlag}" Value="True">

        <Setter Property="Visibility" Value="Visible"/>

        </DataTrigger>

        </Style.Triggers>

        </Style>

        </TextBlock.Style>

        </TextBlock>

        </DataTemplate>

        </DataGridTemplateColumn.CellTemplate>

        </DataGridTemplateColumn>

        <DataGridTextColumn Header="Наименование" Binding="{Binding Name}" Width="*"
MinWidth="180"/>

```

```

        <DataGridTextColumn Header="Описание" Binding="{Binding Description}" Width="*"
MinWidth="200"/>

        <DataGridTemplateColumn Header="Дата" Width="120" SortMemberPath="Date">

            <DataGridTemplateColumn.CellTemplate>

                <DataTemplate>

                    <TextBlock Text="{Binding Date,Converter={StaticResource DTConv}}}"
TextWrapping="Wrap"/>

                </DataTemplate>

            </DataGridTemplateColumn.CellTemplate>

        </DataGridTemplateColumn>

        <DataGridTemplateColumn Header="Статус" Width="110" SortMemberPath="Status">

            <DataGridTemplateColumn.CellTemplate>

                <DataTemplate>

                    <TextBlock Text="{Binding Status,Converter={StaticResource StConv}}}"
TextWrapping="Wrap"/>

                </DataTemplate>

            </DataGridTemplateColumn.CellTemplate>

        </DataGridTemplateColumn>

        <DataGridTemplateColumn Header="Приоритет" Width="110"
SortMemberPath="Priority">

            <DataGridTemplateColumn.CellTemplate>

                <DataTemplate>

                    <TextBlock Text="{Binding Priority,Converter={StaticResource PrConv}}}"
TextWrapping="Wrap"/>

                </DataTemplate>

            </DataGridTemplateColumn.CellTemplate>

        </DataGridTemplateColumn>

    </DataGrid.Columns>

</DataGrid>

</Grid>

</Grid>

</Window>

```

```

ToDo\Windows\TaskListWindow.xaml.cs
using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;
using System.Globalization;
using ToDo.Data.Models;
using ToDo.Helpers;
using ToDo.Data.Services;
namespace ToDo.Windows
{
    /// <summary>
    /// Логика взаимодействия для TaskListWindow.xaml
    /// </summary>
    public partial class TaskListWindow : Window
    {
        private TaskService _service;
        public TaskListWindow(TaskService service)
        {
            InitializeComponent();
            _service = service;
            FillCollection(false);
        }
        private void FillCollection(bool applyFilters)
        {
            int idx = 0;
            if (TasksList.SelectedIndex > 0)
                idx = TasksList.SelectedIndex;
            TasksList.ItemsSource = null;
            TasksList.Items.Clear();

            if (applyFilters)
            {
                TaskFilter filter = new TaskFilter

```

```

    {
        SearchText = SearchField.Text,
        Status = GetStatusFilter(),
        Priority = GetPriorityFilter(),
        Date = DateField.SelectedDate,
        ImportantOnly = ImportantOnlyField.IsChecked.Value,
    };

    TasksList.ItemsSource = _service.Get(filter);
}
else
{
    TasksList.ItemsSource = _service.Get();
}
if (TasksList.Items.Count > 0 && idx < TasksList.Items.Count)
{
    TasksList.SelectedIndex = idx;
}
else
{
    TasksList.SelectedIndex = -1;
}
}

private void FiltersChanged(object sender, EventArgs e)
{
    FillCollection(true);
}

private void NewTask_Executed(object sender, ExecutedRoutedEventArgs e)
{
    var add = new TaskWindow(_service);
    if (add.ShowDialog() == true)
    {

```

```

        FillCollection(true);
    }
}

private void TasksList_MouseDoubleClick(object sender, MouseButtonEventArgs e)
{
    EditTask();
}

private void EditTask_Executed(object sender, ExecutedRoutedEventArgs e)
{
    EditTask();
}

private void DeleteTask_Executed(object sender, ExecutedRoutedEventArgs e)
{
    if (TasksList.SelectedItem is Data.Models.Task selectedTask)
    {
        var res = _service.DeleteTask(selectedTask.Id);
        if (res)
        {
            FillCollection(true);
        }
    }
}

private void EditTask()
{
    if (TasksList.SelectedItem is Data.Models.Task selectedTask)
    {
        var editWindow = new TaskWindow(_service, selectedTask);
        if (editWindow.ShowDialog() == true)
        {
            FillCollection(true);
        }
    }
}

```

```

    }

    private void GetStatistics_Executed(object sender, ExecutedRoutedEventArgs e)
    {
        var stats = _service.GetStatistics();
        var statsWindow = new StatisticsWindow(stats)
        {
            Owner = this
        };
        statsWindow.ShowDialog();
    }

    private void ClearFilters_Executed(object sender, ExecutedRoutedEventArgs e)
    {
        SearchField.Text = string.Empty;
        StatusField.SelectedIndex = 0;
        PriorityField.SelectedIndex = 0;
        DateField.SelectedDate = null;
        ImportantOnlyField.IsChecked = false;
        FillCollection(true);
    }

    private Status? GetStatusFilter()
    {
        var selectedItem = StatusField.SelectedItem as ComboBoxItem;
        if (selectedItem != null)
        {
            var conv = new StatusConverter();
            var result = conv.ConvertBack(selectedItem.Content?.ToString(), typeof(Status), null,
CultureInfo.CurrentCulture);
            return result as Status?;
        }

        return null;
    }
}

```

```

private Priority? GetPriorityFilter()
{
    var selectedItem = PriorityField.SelectedItem as ComboBoxItem;
    if (selectedItem != null)
    {
        var conv = new PriorityConverter();
        var result = conv.ConvertBack(selectedItem.Content?.ToString(), typeof(Priority), null,
CultureInfo.CurrentCulture);
        return result as Priority?;
    }
    return null;
}
}
}

```

ToDo\Windows\StatisticsWindow.xaml

```

<Window x:Class="ToDo.Windows.StatisticsWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Статистика задач" Height="360" Width="360" WindowStartupLocation="CenterOwner"
    ResizeMode="NoResize">
    <Grid Margin="12">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>
        <TextBlock Text="Статистика задач" FontSize="18" FontWeight="Bold" Margin="0 0 0 16" />
        <Grid Grid.Row="1">
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto" />
                <RowDefinition Height="Auto" />
                <RowDefinition Height="Auto" />
            </Grid.RowDefinitions>

```



```

        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
        <RowDefinition Height="Auto" />
    </Grid.RowDefinitions>

    <Grid.ColumnDefinitions>

        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="Auto" />
    </Grid.ColumnDefinitions>

    <TextBlock Grid.Row="0" Grid.Column="0" Text="Всего задач:" Margin="0 4" />
    <TextBlock Grid.Row="0" Grid.Column="1" Text="{Binding TotalTasks}" Margin="16 4 0 4"
    FontWeight="Bold" />
    <TextBlock Grid.Row="1" Grid.Column="0" Text="Новые:" Margin="0 4" />
    <TextBlock Grid.Row="1" Grid.Column="1" Text="{Binding New}" Margin="16 4 0 4"
    FontWeight="Bold" />
    <TextBlock Grid.Row="2" Grid.Column="0" Text="В процессе:" Margin="0 4" />
    <TextBlock Grid.Row="2" Grid.Column="1" Text="{Binding InProgress}" Margin="16 4 0 4"
    FontWeight="Bold" />
    <TextBlock Grid.Row="3" Grid.Column="0" Text="Выполнены:" Margin="0 4" />
    <TextBlock Grid.Row="3" Grid.Column="1" Text="{Binding Completed}" Margin="16 4 0 4"
    FontWeight="Bold" />
    <TextBlock Grid.Row="4" Grid.Column="0" Text="Важные:" Margin="0 4" />
    <TextBlock Grid.Row="4" Grid.Column="1" Text="{Binding Important}" Margin="16 4 0 4"
    FontWeight="Bold" />
    <TextBlock Grid.Row="5" Grid.Column="0" Text="Просрочено:" Margin="0 4" />
    <TextBlock Grid.Row="5" Grid.Column="1" Text="{Binding Expired}" Margin="16 4 0 4"
    FontWeight="Bold" />
    <TextBlock Grid.Row="6" Grid.Column="0" Text="Сегодня:" Margin="0 4" />
    <TextBlock Grid.Row="6" Grid.Column="1" Text="{Binding Today}" Margin="16 4 0 4"
    FontWeight="Bold" />

    <TextBlock Grid.Row="7" Grid.Column="0" Text="До конца недели:" Margin="0 4" />
    <TextBlock Grid.Row="7" Grid.Column="1" Text="{Binding DueThisWeek}" Margin="16 4 0
    4" FontWeight="Bold" />

```

```

        </Grid>

        <Button Grid.Row="2" Content="Закрыть" Width="90" Height="30" HorizontalAlignment="Right"
Margin="0 16 0 0" Click="BtnClose_Click"/>

    </Grid>

</Window>

```

```

ToDo\Windows\StatisticsWindow.xaml.cs
using System.Windows;
using ToDo.Data.Models;
namespace ToDo.Windows
{
    public partial class StatisticsWindow : Window
    {
        public StatisticsWindow(Stats stats)
        {
            InitializeComponent();
            DataContext = stats;
        }

        private void BtnClose_Click(object sender, RoutedEventArgs e)
        {
            Close();
        }
    }
}

```

```

ToDo\Helpers\DateTimeConverter.cs
using System.Globalization;
using System.Windows.Data;
namespace ToDo.Helpers
{
    public class DateTimeConverter : IValueConverter
    {
        public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
        {

```

```

        if (value is DateTime dateTime)
        {
            return dateTime.ToString("dd.MM.yyyy");
        }

        return value;
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }
}

```

ToDo\Helpers\PriorityConverter.cs

```

using System.Globalization;
using System.Windows.Data;
using ToDo.Data.Models;
namespace ToDo.Helpers
{
    class PriorityConverter : IValueConverter
    {
        public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
        {
            if (value is Priority priority)
            {
                return priority switch
                {
                    Priority.Low => "Низкий",
                    Priority.Medium => "Средний",
                    Priority.High => "Высокий",
                    _ => priority.ToString()
                };
            }
        }
    }
}

```

```

        return value;
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        Priority? res = null;

        if (value is Priority pr)
            res = pr;
        if (value is string s)
        {
            res = s switch
            {
                "Низкий" => Priority.Low,
                "Средний" => Priority.Medium,
                "Высокий" => Priority.High,
                _ => null,
            };
        }
        return res;
    }
}

```

ToDo\Helpers\StatusConverter.cs

```

using System.Globalization;
using System.Windows.Data;
using ToDo.Data.Models;
namespace ToDo.Helpers
{
    class StatusConverter : IValueConverter
    {
        public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
        {

```

```

        if (value is Status status)
        {
            return status switch
            {
                Status.New => "Новая",
                Status.InProgress => "В процессе",
                Status.Completed => "Завершена",
                _ => status.ToString()
            };
        }

        return value;
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        Status? res = null;

        if (value is Status st)
        {
            res = st;
        }

        if (value is string s)
        {
            res = s switch
            {
                "Новые" => Status.New,
                "В процессе" => Status.InProgress,
                "Завершены" => Status.Completed,
                _ => null,
            };
        }

        return res;
    }
}

```

```

ToDo.Test\TaskServiceTests.cs
using System;
using System.IO;
using System.Linq;
using ToDo.Data.Context;
using ToDo.Data.Models;
using ToDo.Data.Services;
using Xunit;
namespace ToDo.Test
{
    public class TaskServiceTests : IDisposable
    {
        private readonly string _tempFile;
        private readonly XmlContext _context;
        private readonly TaskService _service;
        public TaskServiceTests()
        {
            ToDo.Data.Models.Task.Counter = 0;

            _tempFile = Path.Combine(Path.GetTempPath(), $"todo_test_{Guid.NewGuid()}.xml");
            _context = new XmlContext(_tempFile);
            _service = new TaskService(_context);

            _context.Tasks.Add(new ToDo.Data.Models.Task { Name = "Task A", Description = "One",
Priority = Priority.Low, Status = Status.New, ImportantFlag = false });
            _context.Tasks.Add(new ToDo.Data.Models.Task { Name = "Task B", Description = "Two",
Priority = Priority.High, Status = Status.InProgress, ImportantFlag = true });
            _context.Tasks.Add(new ToDo.Data.Models.Task { Name = "Task C", Description = "Three",
Priority = Priority.Medium, Status = Status.Completed, ImportantFlag = true });
        }
        [Fact]
        public void Get_NoFilter_ReturnsAll()
        {
            var all = _service.Get();
            Assert.Equal(3, all.Count);
        }
    }
}

```

```

    }

    [Fact]
    public void Get_FilterByPriority_ReturnsMatching()
    {
        var filter = new TaskFilter { Priority = Priority.High };
        var result = _service.Get(filter);

        Assert.Single(result);

        Assert.Equal(Priority.High, result.First().Priority);
    }

    [Fact]
    public void Get_ImportantOnly_ReturnsOnlyImportant()
    {
        var filter = new TaskFilter { ImportantOnly = true };
        var result = _service.Get(filter);

        Assert.Equal(2, result.Count);

        Assert.All(result, t => Assert.True(t.ImportantFlag));
    }

    [Fact]
    public void SaveTask_PersistsAndAdds()
    {
        var newTask = new ToDo.Data.Models.Task { Name = "Saved Task", Description = "saved",
        Priority = Priority.Low, Status = Status.New, ImportantFlag = false };

        var ok = _service.SaveTask(newTask);

        Assert.True(ok);

        Assert.Contains(_context.Tasks, t => t.Name == "Saved Task");

        Assert.True(File.Exists(_tempFile));
    }

    [Fact]
    public void DeleteTask_RemovesExisting()
    {
        var t = new ToDo.Data.Models.Task { Name = "ToDelete", Description = "del", Priority =
        Priority.Low, Status = Status.New, ImportantFlag = false };

        _service.SaveTask(t);
    }

```

```

var id = t.Id;

var res = _service.DeleteTask(id);

Assert.True(res);

Assert.DoesNotContain(_context.Tasks, x => x.Id == id);

```

[Fact]

```

public void EditTask_UpdatesFields()
{
    var t = new ToDo.Data.Models.Task { Name = "EditMe", Description = "orig", Priority =
Priority.Low, Status = Status.New, ImportantFlag = false };

    _service.SaveTask(t);

    t.Name = "Edited";

    t.Description = "Changed";

    t.Priority = Priority.High;

    var res = _service.EditTask(t);

    Assert.True(res);

    var updated = _context.Tasks.First(x => x.Id == t.Id);

    Assert.Equal("Edited", updated.Name);

    Assert.Equal("Changed", updated.Description);

    Assert.Equal(Priority.High, updated.Priority);
}

```

[Fact]

```

public void Get_FilterByStatus_ReturnsMatching()
{
    var filter = new TaskFilter { Status = Status.InProgress };

    var result = _service.Get(filter);

    Assert.Single(result);

    Assert.Equal(Status.InProgress, result.First().Status);
}

```

[Fact]

```

public void Get_FilterByDate_ReturnsMatching()
{

```



```

        var targetDate = new DateTime(2026, 6, 21);

        var t = new ToDo.Data.Models.Task { Name = "DateTask", Description = "d", Priority =
Priority.Low, Status = Status.New, ImportantFlag = false, Date = targetDate };

        _context.Tasks.Add(t);

        var filter = new TaskFilter { Date = targetDate };

        var result = _service.Get(filter);

        Assert.Contains(result, x => x.Date == targetDate);
    }

    [Fact]
    public void Get_SearchText_ReturnsMatching()
    {
        var filter = new TaskFilter { SearchText = "Task A" };

        var result = _service.Get(filter);

        Assert.Single(result);

        Assert.Contains("Task A", result.First().Name);
    }

    public void Dispose()
    {
        try
        {
            if (File.Exists(_tempFile)) File.Delete(_tempFile);
        }
        catch { }
    }
}
}

```

```

ToDo.Data\Context\XmlContext.cs
using System.Collections.ObjectModel;
using System.Xml.Serialization;
namespace ToDo.Data.Context
{
    public class XmlContext

```

```

{
    public List<Models.Task> Tasks;
    private readonly string _filePath;
    public XmlContext(string newFilePath)
    {
        Tasks = new List<Models.Task>();
        _filePath = newFilePath;
        FillTasks();
    }
    private void FillTasks()
    {
        if (!File.Exists(_filePath))
            return;
        try
        {
            var seializer = new XmlSerializer(typeof(List<Models.Task>));
            using var stream = File.OpenRead(_filePath);
            var loadedTasks = (List<Models.Task>?)serializer.Deserialize(stream);
            if (loadedTasks != null && loadedTasks.Any())
            {
                Tasks = new List<Models.Task>(loadedTasks);
                var maxId = Tasks.Max(t => t.Id);
                if (maxId >= Models.Task.Counter)
                {
                    Models.Task.Counter = maxId + 1;
                }
            }
        }
        catch (InvalidOperationException)
        {
        }
    }
    public void WriteToFile()

```

```

    {
        var serializer = new XmlSerializer(typeof(List<Models.Task>));
        var tasksToSave = Tasks.ToList();
        using var stream = File.Open(_filePath, FileMode.Create, FileAccess.Write, FileShare.None);
        serializer.Serialize(stream, tasksToSave);
    }
}
}

```

ToDo.Data\Services\TaskService.cs

```

using System;
using System.Collections.ObjectModel;
using System.Linq;
using ToDo.Data.Context;
using ToDo.Data.Models;
namespace ToDo.Data.Services
{
    public class TaskService
    {
        private XmlContext _context;
        public TaskService(XmlContext context)
        {
            if (context != null)
            {
                _context = context;
            }
        }

        public ObservableCollection<Models.Task> Get()
        {
            return new ObservableCollection<Models.Task>(_context.Tasks);
        }

        public ObservableCollection<Models.Task> Get(TaskFilter filter)
        {

```

```

var searchText = filter.SearchText?.ToLower();
var filteredTasks = _context.Tasks
    .Where(x => (string.IsNullOrEmpty(searchText)
        || x.Name.ToLower().Contains(searchText)
        || (!string.IsNullOrEmpty(x.Description) &&
x.Description.ToLower().Contains(searchText)))
        && (!filter.Id.HasValue || x.Id == filter.Id.Value)
        && (!filter.Status.HasValue || x.Status == filter.Status.Value)
        && (!filter.Priority.HasValue || x.Priority == filter.Priority.Value)
        && (!filter.Date.HasValue || x.Date == filter.Date.Value)
        && (!filter.ImportantOnly || x.ImportantFlag))
    .ToList();

return new ObservableCollection<Models.Task>(filteredTasks);
}

```

```

public bool SaveTask(Models.Task newTask)
{
    if (newTask == null || string.IsNullOrEmpty(newTask.Name))
    {
        return false;
    }

    _context.Tasks.Add(newTask);
    _context.WriteToFile();

    return true;
}

```

```

public bool DeleteTask(int id)
{
    Models.Task? target = _context.Tasks
        .FirstOrDefault(t => t.Id == id);

    if (target == null)
    {
        return false;
    }
}

```

```

    }

    _context.Tasks.Remove(target);

    _context.WriteToFile();

    return true;
}

public bool EditTask(models.Task task)
{
    models.Task? target = _context.Tasks
        .FirstOrDefault(t => t.Id == task.Id);

    if (target == null)
    {
        return false;
    }

    target.Priority = task.Priority;
    target.Description = task.Description;
    target.Status = task.Status;
    target.Date = task.Date;
    target.Name = task.Name;

    _context.WriteToFile();

    return true;
}

public Stats GetStatistics()
{
    var tasks = _context.Tasks.ToList();

    var stats = new Stats
    {
        TasksCount = tasks.Count,
        New = tasks.Count(t => t.Status == Status.New),
        InProgress = tasks.Count(t => t.Status == Status.InProgress),
        Completed = tasks.Count(t => t.Status == Status.Completed),
        Important = tasks.Count(t => t.ImportantFlag),
    }
}

```

```

        Expired = tasks.Count(t => t.Date.Date < DateTime.Now.Date && t.Status !=
Status.Completed),

        Today = tasks.Count(t => t.Date.Date == DateTime.Today),

        DueThisWeek = tasks.Count(t => t.Date <= DateTime.Today.AddDays(7)),

    };

    return stats;

}

}

}

```

ToDo.DataModels\Priority.cs

```

using System;

using System.Collections.Generic;

using System.Text;

namespace ToDo.Data.Models
{
    public enum Priority
    {
        Low = 0,
        Medium = 1,
        High = 2,
    }
}

```

ToDo.Data\Models\Status.cs

```

using System;

using System.Collections.Generic;

using System.Text;

namespace ToDo.Data.Models
{
    public enum Status
    {
        New = 0,
        InProgress = 1,

```

```

        Completed = 2,
    }
}

```

ToDo.Data\Models\Task.cs

```

using System;
using System.Collections.Generic;
using System.Net.NetworkInformation;
using System.Text;
namespace ToDo.Data.Models
{
    public class Task
    {
        public static int Counter = 0;
        public int Id { get; set; }
        public string Name { get; set; }
        public string Description { get; set; }
        public Priority Priority { get; set; }
        public DateTime Date { get; set; }
        public Status Status { get; set; }
        public bool ImportantFlag { get; set; }
        public Task()
        {
            Id = ++Counter;
            Priority = Priority.Low;
            Status = Status.New;
            Date = DateTime.Now.Date;
            ImportantFlag = false;
        }
    }
}

```

ToDo.Data\Models\TaskFilter.cs

```

using System;

```

```

using System.Collections.Generic;
using System.Text;
namespace ToDo.Data.Models
{
    public class TaskFilter
    {
        public int? Id { get; set; }
        public string SearchText { get; set; }
        public Priority? Priority { get; set; }
        public DateTime? Date { get; set; }
        public Status? Status { get; set; }
        public bool ImportantOnly { get; set; }
    }
}

```

```

ToDo.Data\Models\Stats.cs
using System;
using System.Collections.Generic;
using System.Text;
namespace ToDo.Data.Models
{
    public class Stats
    {
        public int TasksCount { get; set; }
        public int New { get; set; }
        public int InProgress { get; set; }
        public int Completed { get; set; }
        public int Important { get; set; }
        public int Expired { get; set; }
        public int Today { get; set; }
        public int DueThisWeek { get; set; }
    }
}

```